Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

«Поисковый робот»

Студент: В. О. Горячкин
Преподаватель: А. А. Кухтичев
Группа: М8О-407Б-23
Дата: 21.10.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2
«Поисковый робот»

Необходимо написать поисковый робот — компоненту обкачки документов.

- Единственным аргументом роботу подаётся путь до yaml-конфига, содержащего данные для базы данных в секции `db`, данные для робота в секции `logic` (задержка между обкачкой страницы) и любые другие данные, необходимые для реализации логики робота.
- Сохранять в базе данных (например, MongoDB) документы с полями: нормализованный url, «сырой» html-текст документа, название источника, дата обкачки в формате Unix timestamp.
- Робот можно остановить в любой момент, и при повторном запуске он должен начать с того документа, на котором остановился.
- Периодически робот должен уметь переобкачивать документы, которые уже есть в базе, но только если они изменились.

1 Устройство робота

Робот обкачивает два источника корпуса «кино и сериалы»: русскую Википедию (статьи о фильмах, сериалах и людях кино) и film.ru (рецензии, подборки, интервью). Работа состоит из двух программ. Обе принимают единственный аргумент — путь до общего yaml-конфига.

- `src/crawler.py` — обкачка: берёт URL из очереди, скачивает документ, выделяет текст, сохраняет в базу и пополняет очередь найденными ссылками;
- `src/recrawler.py` — переобкачка: проверяет, не изменились ли уже скачанные документы, и изменившиеся ставит в очередь.

Ещё три утилиты ничего не качают из сети: `src/stats.py` печатает сводку по базе, `src/validate.py` проверяет, что в ней лежит то, что должно (раздел 4), а `src/reparse.py` заново выделяет текст из сохранённого `raw_html`, когда меняются правила разбора.

Общий код разнесён по модулям, у каждого одна ответственность:

Модуль	**Что делает**
`config.py`	чтение yaml-конфига
`urls.py`	нормализация URL
`parsing.py`	определение кодировки, выделение текста, ссылок и заголовка, отпечаток текста
`rules.py`	фильтры источника: что обходить, откуда собирать ссылки, что сохранять
`fetcher.py`	HTTP-клиент: паузы вежливости, `robots.txt`, условные запросы
`storage.py`	MongoDB: очередь URL и документы
`shutdown.py`	мягкая остановка по `SIGINT`/`SIGTERM`
`stats.py`	сводка по базе, отдельная утилита
`validate.py`	проверка содержимого базы, отдельная утилита
`reparse.py`	переразбор текста из сохранённого HTML без перекачки

Хранилище и HTTP-клиент передаются в робота и переобкачку снаружи. В рабочем режиме это настоящие `Storage` и `Fetcher`, в автотестах — временная база и подменённая сеть (раздел 4).

Хранилище

В качестве базы данных выбрана MongoDB — её называет и задание. Документная модель здесь естественна: документ корпуса — это запись с набором полей, без связей

между таблицами, а схема легко расширяется новыми полями без миграций. База поднимается в докере одной командой (`make db-up`), данные лежат в именованном томе и переживают пересоздание контейнера. Адрес сервера и имя базы задаются в секции `db` конфига.

В базе две коллекции. `documents` хранит сами документы:

- `url` — нормализованный адрес, уникальный индекс;
- `raw_html` — «сырой» html-текст документа, как пришёл;
- `text` — выделенный текст без навигационной обвязки;
- `source` — название источника;
- `crawled_at` — дата обкачки, Unix timestamp;
- `content_hash` — отпечаток текста, нужен переобкачке;
- `etag`, `last_modified` — для условных запросов;
- `revision` — сколько раз документ скачивался;
- `raw_size`, `text_size`, `text_bytes` — размеры HTML и текста (в символах и в байтах), чтобы статистика не читала тела документов.

Задание требует хранить «сырой» HTML, а лекция — только необходимый текст. Требования не противоречат друг другу, если хранить и то, и другое: `raw_html` пригодится, если правила выделения текста изменятся, а переразбирать корпус по ним, не выкачивая его заново, гораздо дешевле.

Вторая коллекция, `frontier`, — это очередь URL: адрес, источник, глубина, статус (`pending`, `done`, `failed`, `skipped`), приоритет, число попыток и текст ошибки. Индекс `(status, priority, _id)` делает выбор следующего адреса одним запросом по индексу. Порядок внутри приоритета — по `_id`: `ObjectId` начинается с метки времени, так что это порядок добавления.

Ссылки со страницы добавляются в очередь одной пачкой, `insert_many` без упорядочивания: уже известные адреса отбивает уникальный индекс, остальные вставляются. Документ сохраняется одной операцией `find_one_and_update` с `upsert`: она же возвращает прежний отпечаток, по которому видно, изменился ли текст.

Остановка и продолжение

Очередь лежит в базе, а не в памяти. URL помечается как `done` только после того, как документ сохранён, поэтому прерывание в любой момент безопасно: недообработанный адрес остаётся в статусе `pending` и при следующем запуске будет взят снова. Сигналы `SIGINT` и `SIGTERM` перехватываются — робот дописывает текущий документ и выходит, а не умирает посреди записи.

На полной обкачке это пригодилось на деле: робот останавливали вручную, ноутбук засыпал и один раз перезагрузился. Каждый раз следующий запуск продолжал с того же места.

Уникальность `url` в обеих коллекциях снимает вопрос дублирования ссылок: повторное добавление просто игнорируется базой.

Нормализация URL

Часть ссылок на странице относительные, поэтому при разборе они склеиваются с адресом текущей страницы. HTML-сущности в атрибуте `href` раскрываются (`&` — это `&`). Дальше из адреса убирается всё, что не влияет на содержимое: якорь, служебные параметры (`utm_*`, `gclid` и подобные), порт по умолчанию, регистр схемы и хоста; параметры запроса сортируются.

Отдельно пришлось раскрывать процентные последовательности. Ссылки в HTML Википедии закодированы, и адрес `/wiki/%D0%A4%D0%B8%D0%BB%D1%8C%D0%BC` — это тот же документ, что и `/wiki/Фильм`. Без раскрытия в базе появлялись бы два документа вместо одного. Не раскрываются только `%25` (сам знак процента) и экранированные разделители `%2F`, `%3F`, `%23`: раскрыв их, получим другой адрес.

Повторные слэши в пути сохраняются. Первая версия их схлопывала, и статьи с названиями вроде `.hack//SIGN` получали несуществующий адрес и ответ 404.

Определение кодировки

Порядок ровно такой, как на лекции: сначала заголовок `Content-Type`, затем `<meta charset>` в самом HTML, и только потом UTF-8 по умолчанию. Оба итоговых источника отдают UTF-8, но первоначальный второй источник, Кино-Театр.Ру, отдавал windows-1251, и без определения кодировки текст с него попадал бы в базу нечитаемым. Этот случай закреплён в автотестах на сохранённых страницах сайта.

Вежливость

Робот соблюдает две вещи. Явные правила — `robots.txt`: он скачивается для каждого хоста и кэшируется, запрещённые адреса помечаются как `skipped`. Неявные — пауза между запросами к одному хосту: общая задаётся в секции `logic` конфига (1 секунда), у источника может быть своя (у film.ru — 2 секунды, сайт меньше Википедии).

Выбор следующего URL

Очередь отдаёт сначала адреса, которые ждут переобкачки, из любого источника. Остальные — в порядке добавления, то есть обход идёт в ширину.

Из какого источника взять новый адрес, робот решает по готовности хоста: берётся тот, у которого пауза вежливости истекла раньше, при равенстве — тот, к которому дольше не обращались. Это упрощённая версия очереди на каждый хост из схемы URL frontier.

Первая версия просто чередовала источники, и это оказалось ошибкой. У одного источника пауза была 5 секунд, у Википедии — одна, и при строгом чередовании каждый второй запрос ждал пять секунд: Википедия качалась со скоростью медленного источника, около 0,4 страницы в секунду. С выбором по готовности медленный хост получает запрос раз в несколько секунд, а в промежутках робот качает быстрый.

Сбои сети и блокировки

Ошибки скачивания делятся на два вида. Настоящие (`404` и другие ответы 4xx) означают, что страницы нет, — адрес сразу помечается `failed`. Временные — нет соединения, таймаут, ошибка DNS, ответы 5xx и 429 — говорят о том, что страница, скорее всего, есть, но сейчас недоступна. Для них адрес возвращается в очередь, а хост ставится на паузу: 30 секунд, затем вдвое дольше, но не больше 10 минут. Пометку `failed` адрес получает только после пяти таких сбоев подряд.

Так же обрабатывается капча: если в тексте страницы найден признак заглушки (`blocked_marker` в конфиге источника), страница не сохраняется, источник встаёт на паузу от 10 минут до 2 часов, а адрес ждёт повторной попытки.

Первая версия любую ошибку считала окончательной, и это стоило дорого: за несколько часов без интернета робот «пролистал» очередь и списал как недоступные 12 тысяч адресов, которые на деле были в порядке (раздел «Результаты»).

Выделение текста

Навигационная обвязка удаляется за счёт того, что текст берётся только из основного контейнера страницы. Контейнер задаётся в конфиге источника: у Википедии это `<div class="mw-content-ltr mw-parser-output">`, у film.ru — `<div class="wrapper_articles_text">`. Граница контейнера находится по балансу вложенных `<div>`: всё, что после парного `</div>`, — боковые панели, комментарии и подвал сайта. Внутри контейнера выбрасываются скрипты, стили и таблицы (инфобоксы), затем снимаются теги, а HTML-сущности раскрываются (`«` — это «», `&` — «&»). Хвост документа отрезается по маркерам `cut_after`: у Википедии это разделы «Примечания», «Литература» и «Ссылки», у film.ru — подпись автора и кнопки «Поделиться».

Фильтры ссылок

Фильтров три, и это оказалось важным. Первый, `allow`/`deny`, решает, обходить ли ссылку вообще. Второй, `store_allow` и `store_deny`, решает, сохранять ли страницу как документ.

Разделять их пришлось потому, что каталоги, списки и страницы категорий нужны роботу только как источник ссылок. Текста в них хватает, чтобы пройти фильтр по длине, и в первой версии робота база наполнилась оглавлениями вместо статей.

Третий, `follow_only`, — список страниц, с которых вообще собираются ссылки. Без него робот уходит из своей тематики: из статьи о фильме ссылки ведут на произвольные статьи Википедии, и через несколько шагов корпус перестаёт быть кинематографическим.

Как устроен обход источников

Википедия. Документы ищутся через категории: робот собирает ссылки только со страниц категорий, а статьи сохраняет. Сначала стартовыми были алфавитные категории («Фильмы по алфавиту» и подобные), но страница категории показывает только первые 200 статей. Продолжение лежит по адресам `/w/index.php?...&pagefrom=...`, а весь путь `/w/` закрыт в `robots.txt` Википедии. Поэтому корпус набирается иначе: робот обходит всё дерево категорий о кино — по годам, странам, жанрам и их пересечениям («Фильмы Франции 1995 года»). Категорий тысячи, из каждой берутся первые 200 статей, и в сумме это десятки тысяч документов.

Чтобы обход не ушёл в книги и картины, разрешены только категории, в названии которых есть «фильм», «сериал», «актёр», «актриса» или «кинорежиссёр»; служебные и мета-категории («Википедия:...», «Категории_...») отсекаются. Отсекаются

и адреса Википедии с любыми параметрами: это ссылки на правку, историю и «красные ссылки» на несуществующие страницы (`?action=edit&redlink=1`).

film.ru. Статьи собраны в три списка — рецензии, подборки, интервью, по 28 статей на странице. Списки листаются через ajax-запрос, который возвращает JSON, а ссылок на `?page=N` в HTML нет. Поэтому страницы списков задаются в конфиге шаблоном `seed_pages`: `https://www.film.ru/topic/review?page={n}` для n от 0 до последней страницы. Номер последней страницы найден перебором: следующая отвечает 404. Робот обходит эти 545 страниц и сохраняет только сами статьи (`/articles/...`); короткие карточки фильмов документами не считаются.

2 Переобкачка

Переобкачка вынесена в отдельную программу, `src/recrawler.py`. Она ничего не скачивает *в базу*: её задача — определить, изменился ли документ, и если да, поставить его URL в очередь. Дальше документ заберёт обычный робот.

Программа берёт из базы документы, которые давно не проверялись (порог задаётся в секции `recrawl` конфига), и по каждому идёт в два шага — от дешёвого к дорогому.

Периодичность обеспечивает системный планировщик, а не сама программа: например, строка в `crontab`

```
0 3 * * *  cd ~/MAI-IR/labs/lab-02 && make recrawl
```

запускает проверку каждую ночь. Порог `min_age_hours` не даёт проверять один документ чаще раза в сутки, сколько бы раз программу ни запустили, а `batch` ограничивает число запросов за один запуск.

Шаг 1. Условный запрос. Если у документа сохранены `ETag` или `Last-Modified`, запрос отправляется с заголовками `If-None-Match` и `If-Modified-Since`. Ответ `304 Not Modified` означает, что документ точно не менялся, — причём тело страницы сервер при этом не передаёт вовсе.

Шаг 2. Сравнение отпечатка. Если сервер условные запросы не поддерживает, страница скачивается, из неё выделяется текст и считается отпечаток SHA-256. Он сравнивается с сохранённым в базе.

Сравнивается именно *выделенный текст*, а не «сырой» HTML. Это принципиально: в HTML на каждой перезагрузке меняются счётчики, идентификаторы сессий, рекламные блоки и метки времени, и по нему почти любой документ выглядел бы изменившимся. Отпечаток текста меняется только тогда, когда изменилось содержание.

Изменившийся URL попадает в очередь со статусом `pending` и приоритетом 1. Приоритет важен: очередь отдаёт такие адреса раньше новых, поэтому обновление уже известных документов не ждёт, пока разгребётся весь фронт обкачки. При повторной обкачке запись в `documents` обновляется, а счётчик `revision` увеличивается на единицу.

Проверка работы

Цикл проверялся на настоящих данных дважды.

Первая проверка, 7 октября. Через неделю после демонстрационного прогона (раздел «Результаты») все 100 документов тогдашней базы стали старше суток, и переобкачка проверила их целиком:

```
[изменился] https://ru.wikipedia.org/wiki/7_ящиков
[изменился] https://ru.wikipedia.org/wiki/9_песен
...
Время работы:              113.7 с
Проверено документов:      100
ответ 304 Not Modified:  68
текст совпал:            24
изменились:              8
В очередь на переобкачку добавлено: 8
```

Две трети документов отсеялись первым, дешёвым шагом: Википедия поддерживает условные запросы, и на `304` тело страницы не передаётся. У 24 документов сервер отдал страницу целиком, но выделенный текст не изменился, — это ровно тот случай, ради которого сравнивается текст, а не HTML: по HTML все 24 считались бы изменившимися.

Восемь изменений проверены вручную, и все они настоящие правки статей. В «7 ящиках» появились разделы о критике и наградах. Из «9 песен» и «8 женщин» редакторы удалили разделы с плашкой «В разделе не хватает ссылок на источники». В остальных поправлены отдельные слова, цифры и типографика.

Запущенный следом робот взял все восемь адресов раньше новых и поднял им `revision` с 1 до 2:

```
[updated ]    2214 симв.  https://ru.wikipedia.org/wiki/7_ящиков
[updated ]    2766 симв.  https://ru.wikipedia.org/wiki/9_песен
...
```

Эта проверка нашла и ошибку. Сначала робот перекачал один адрес и переключился на новые страницы Кино-Театр.Ру: после запроса к Википедии её хост был занят паузой вежливости, а выбор по готовности хоста ставил готовый источник выше приоритета переобкачки. Теперь очередь сначала ищет адреса, ждущие переобкачки, во всех источниках, и только потом — новые; на это добавлен тест.

Вторая проверка, 19 октября, — уже на полном корпусе. Переобкачка взяла 100 документов, которые дольше всех не проверялись (это те же статьи, что и в первый раз):

```
Проверено документов:      100
ответ 304 Not Modified:  0
текст совпал:            97
изменились:              3
```

На этот раз ни один документ не отсеялся условным запросом. Причина не в роботе: 14–15 октября Википедия обновила у всех этих статей дату `Last-Modified`, хотя текст не менялся, — по-видимому, массово правились шаблоны или категории. Сервер честно отдал страницы целиком, и второй шаг, сравнение отпечатков текста, отсеял 97 неизменившихся. Это хорошо показывает, зачем нужны оба шага: условный запрос дешёвый, но полагаться на одну дату изменения нельзя. Новые даты сохранены, так что при следующей проверке эти документы ответят `304`. Три изменившиеся статьи робот перекачал первыми.

3 Результаты

Робот прошёл три этапа. Первый прогон, на ранней версии с хранилищем SQLite, собрал 400 документов и выявил блокировку тогдашнего второго источника, КиноТеатр.Ру. Демонстрационный прогон на MongoDB, до 100 документов, проверил все механизмы на настоящих сайтах и измерил скорость. Наконец, полная обкачка 15–19 октября собрала итоговый корпус. Цифры корпуса ниже взяты из базы в её нынешнем виде; их же показывают `make stats` и `make validate`.

Итоговый корпус

Показатель	**Википедия**	**film.ru**	**Всего**
Документов	64 594	14 972	79 566
Объём «сырых» данных	11,6 ГБ	1,6 ГБ	13,2 ГБ
Объём выделенного текста	672 МБ	169 МБ	841 МБ
Доля полезного текста	5,7 %	10,5 %	6,2 %
Средний размер документа	188 КБ	110 КБ	174 КБ
Средний объём текста, символов	6 423	6 530	6 443
Медиана объёма текста, символов	3 629	5 141	—
90-й перцентиль, символов	13 404	11 797	—
Максимум, символов	202 052	87 623	—

Таблица 1: Итоговый корпус

Требование в 30 тысяч документов перекрыто в два с половиной раза. Объём текста считается в байтах UTF-8, доля — как в первой лабораторной работе: байты текста к байтам HTML.

Источники заметно различаются по форме распределения. У Википедии медиана почти вдвое меньше среднего: большинство статей о фильмах короткие, а среднее тянут вверх немногочисленные огромные статьи — об известных людях (Дэвид Боуи, Пол Маккартни, Аль Пачино) и франшизах, по 100–140 тысяч символов. Максимум, 200 тысяч, — у статьи «Россия», одной из немногих посторонних (см. ниже). У film.ru статьи ровнее — медиана близка к среднему, а самые короткие (10-й перцентиль) почти вдвое длиннее, чем у Википедии. Минимум у обоих источников — около 1 500 символов: это порог `min_text_length`.

Тематичность корпуса проверена по категориям, которые Википедия перечисляет в HTML каждой статьи. У 494 статей (0,8 %) нет ни одной категории о кино. Большая часть из них всё равно смежная с темой — компьютерные игры и песни бондианы, сказка о Пиноккио и персонажи франшиз, статьи о кинопроизводстве. Но попали и совсем посторонние — «Россия», «Канада», «Франция»: тематические категории

вида «Фильмы о… » иногда включают и основную статью своей темы.

MongoDB хранит 14,0 ГБ данных в 4,5 ГБ на диске: движок WiredTiger по умолчанию сжимает коллекции, и «сырой» HTML сжимается втрое.

Ход полной обкачки

Полная обкачка заняла четыре дня календарного времени в семи запусках. Робота останавливали вручную, чтобы обновить код, а ноутбук засыпал и один раз перезагрузился; после каждого перерыва робот продолжал с того же места. Обкачка закончилась сама: очередь опустела.

Итоговое состояние очереди — 125 228 обработанных адресов. Из них около 25 тысяч — страницы категорий Википедии и 545 — страницы списков film.ru, которые нужны только ради ссылок. Ещё 6 388 адресов помечены `failed`: почти все это несуществующие категории вроде «Фильмы Дании 1952 года», на которые ссылаются навигационные шаблоны-сетки «страна × год». Они проверены выборочно: Википедия отвечает «нет страницы с таким названием», статей в них нет. 554 адреса пропущены: 539 — ссылки на правку страниц, 9 запрещены `robots.txt`.

Средняя скорость в последнем, самом длинном запуске — 0,51 страницы в секунду. Паузы вежливости позволяют до полутора запросов в секунду на два источника; остальное съедает ожидание ответа сервера. Кроме того, примерно каждый пятый запрос уходит на страницу категории, которая документом не становится.

Что пошло не так

Полная обкачка вскрыла несколько проблем, которых не было видно на сотне документов. Каждая исправлена, и на каждую добавлен тест.

Продолжение категорий закрыто robots.txt. Страница категории Википедии показывает только 200 статей, а ссылки на следующие страницы ведут в `/w/index.php`, который запрещён роботам. Первая версия брала только первые 200 статей каждой алфавитной категории, и очередь кончилась на 1 400 документах. Решение — обход всего дерева категорий о кино (раздел «Устройство робота»).

Уход из тематики через родительские категории. Со страниц категорий фильмов есть ссылки на «Категории произведений искусства по алфавиту» и «Алфавитные категории, наполняемые шаблонами», а через них — на книги, картины и вообще всё. Решение — разрешать только категории со словами о кино в названии.

Блокировка Кино-Театр.Ру. Сайт снова включил капчу — после 549 нормально скачанных страниц, хотя пауза теперь была 5 секунд, а не одна. Значит, защита считает общее число запросов, а не их частоту. Робот капчу распознал и в базу не пустил, но продолжал запрашивать сайт каждые 5 секунд и списал как недоступные 1 606

адресов. После этого появилась пауза источника с нарастанием (раздел «Устройство робота»). Блокировка не снималась больше суток, и за это время с сайта удалось сохранить всего 53 документа. Источник заменён на film.ru: он отдаёт обычный HTML, и за 15 тысяч запросов к нему не было ни одной ошибки.

Обрыв интернета. Ночью несколько часов не работал DNS. Робот считал каждую ошибку окончательной и за это время пометил `failed` 12 320 исправных адресов. Их пришлось вернуть в очередь вручную. После этого временные ошибки сети стали отдельным случаем с паузой и повторами, и дальше робот переживал сбои сам: из 41 сетевого сбоя только два адреса не скачались за пять попыток подряд, а после возврата в очередь скачались и они.

Красные ссылки. Навигационные шаблоны Википедии ссылаются на несуществующие страницы, и такие ссылки ведут на страницу их создания (`?action=edit&redlink=1`). Робот обошёл около 6 900 таких адресов впустую, а одна служебная страница «Создание: …» даже попала в базу как документ. Решение — отсекать адреса Википедии с параметрами; лишний документ удалён.

Двойной слэш в названии. Нормализация схлопывала повторные слэши, и статьи вроде `.hack//SIGN` получали несуществующий адрес. Повторные слэши больше не трогаются.

Демонстрационный прогон и скорость

Демонстрационный прогон на 100 документов (до замены второго источника) сохранил их за 213 секунд при 170 скачанных страницах. На нём была найдена проблема чередования источников: при строгом чередовании скорость была 0,39 страницы в секунду, с выбором по готовности хоста — 0,80, вдвое выше.

Первый прогон и блокировка Кино-Театр.Ру

Первый прогон, на версии с SQLite, дал 400 документов за 933 обойдённых URL. Из 119 адресов документов Кино-Театр.Ру сохранились лишь 17: остальные вернули не документ, а страницу одинакового размера в 1041 символ:

```
Кино-Театр.Ру Роботам запрещено посещать наш сайт.
Пожалуйста,подтвердите,что вы не робот. Проверка: введите...
```

Сайт закрыт сервисом ddos-guard, и после примерно 470 запросов защита перевела робота на капчу. Хуже всего то, что робот подмены не заметил: ответ приходил с кодом 200, и страница с капчой выглядела как любой другой документ. От попадания в базу ста с лишним одинаковых заглушек спас случай — порог

`min_text_length` в 1 500 символов. Отсюда в роботе появился признак страницы-заглушки `blocked_marker` в конфиге источника. Окончательно вопрос со вторым источником решила только замена на film.ru.

4 Тестирование

Автотесты написаны на `unittest` из стандартной библиотеки и лежат в каталоге `tests/`. Все они работают без интернета: HTTP-клиент подменяется классом `FakeFetcher`, который отдаёт страницы из словаря и, как настоящий сервер, отвечает `304`, если `ETag` совпал. Для разбора используются настоящие страницы из образцов первой лабораторной работы, а фильтры проверяются на рабочем `config.yaml`.

База запускается в двух режимах. `make test` — MongoDB в памяти (`mongomock`): не нужны ни сеть, ни докер, тесты идут за десятые доли секунды. `make test-mongo` — те же тесты на настоящей MongoDB из докера: каждый тест получает свою временную базу со случайным именем и удаляет её после себя. Второй режим нужен, потому что эмулятор повторяет MongoDB не полностью — например, он падал на пакетной вставке через `bulk_write`, которая на настоящем сервере работала. Приложенный результат получен на настоящей MongoDB.

План тестирования

План построен по требованиям задания и по механизмам робота: на каждое требование — хотя бы один тест, который упадёт, если требование нарушено.

Что проверяется	**Как**	**Тестов**
Нормализация URL (`test_urls`)	склейка относительных ссылок, отбрасывание якоря и служебных параметров, раскрытие процентных последовательностей, но не `%25` и разделителей, регистр, порт, повторные слэши, отказ от `mailto:`/`javascript:`, идемпотентность	10
Кодировка (`test_parsing`)	приоритет `Content-Type` над `<meta>`, `<meta charset>` и `http-equiv`, откат на UTF-8, неизвестная кодировка, настоящая страница в windows-1251	6
Выделение текста (`test_parsing`)	баланс вложенных `div`, порядок атрибутов, выбор контейнера по `itemprop`, удаление скриптов и таблиц, `cut_after`; на настоящих страницах Википедии, film.ru и Кино-Театр.Ру — отсутствие меню, подвала, подписи автора и кнопок; раскрытие HTML-сущностей	12

Что проверяется	**Как**	**Тестов**
Ссылки, заголовок, отпечаток (`test_parsing`)	нормализация и дедупликация ссылок, HTML-сущности в `href`, ссылки настоящей страницы, заголовок, SHA-256	5
Фильтры (`test_rules`)	на рабочем конфиге. Википедия: обходятся только категории о кино, мета-категории и адреса с параметрами (правка, красные ссылки, `/w/`) отсекаются, категории не сохраняются, со статей ссылки не собираются. film.ru: статьи сохраняются, страницы списков обходятся, прочее отсекается, на настоящей странице списка находятся все 28 статей. Признак капчи	13
Хранилище (`test_storage`)	уникальность в очереди, порядок выдачи, выбор источника, приоритет переобкачки важнее выбора источника, пакетная вставка, уникальный индекс, ревизии, обязательные поля документа, выбор документов для проверки без тел, очередь переживает переподключение к базе; переразбор текста из `raw_html` без перекачки	14
Робот целиком (`test_crawler`)	полный обход: сохраняется только документ, служебная и короткая страницы отсеяны; в базе только текст контейнера; `max_depth`, `max_documents`; `robots.txt`; ошибка 404; **обрыв сети — повтор с паузой**, долгий обрыв — `failed` после пяти попыток; капча — пауза источника с нарастанием и повтор; шаблон стартовых страниц; **остановка и продолжение с того же URL**; повторный запуск без дублей	13
Выбор источника (`test_crawler`)	хост с неистёкшей паузой не ждём, а берём другой источник; готовые источники чередуются	2

Что проверяется	Как	Тестов
Переобкачка целиком (`test_crawler`)	`304` по `ETag`; изменилась разметка, но не текст — документ не трогаем; изменился текст — URL в очереди с приоритетом, робот перекачивает его первым и поднимает `revision`; недавно проверенные документы не запрашиваются	4
	Всего	**79**

Остановка проверяется так: флаг остановки срабатывает после первого URL, затем к той же базе подключается новый экземпляр робота. Тест требует, чтобы второй запуск запросил ровно оставшиеся адреса и не запросил повторно уже обработанный.

Результат

```
$ make test-mongo
...
test_stop_and_resume (tests.test_crawler.CrawlerTest....) ... ok
...
test_queue_survives_reopen (tests.test_storage.PersistenceTest...) ... ok
----------------------------------------------------------------------
Ran 79 tests in 3.351s

OK
```

Полный вывод — в файле `report/tests.log` (`make test-report`).

Проверка базы

Тесты проверяют код, но не то, что лежит в базе после настоящего прогона. Для этого есть отдельная утилита, `make validate`. По каждому документу она проверяет:

- все обязательные поля на месте, `crawled_at` — целое число секунд в разумных пределах, источник есть в конфиге;
- адрес уникален, нормализован (повторная нормализация его не меняет) и проходит фильтры своего источника;

- выделенный текст заново получается из сохранённого `raw_html` символ в символ, а `content_hash` равен SHA-256 текста, — значит, текст взят именно из этой страницы;
- текст не короче порога, капчи среди документов нет;
- каждый документ отмечен в очереди как обработанный.

С ключом `LIVE=N` утилита ещё перекачивает N случайных документов с сайтов и сравнивает с базой — так видно, что документы настоящие и доступны. На итоговом корпусе все проверки пройдены:

```
Документов: 79566,очередь: {'skipped': 554,'failed': 6388,'done': 125228}
[ok  ] все обязательные поля на месте: 79566/79566
[ok  ] url уникальны: 79566/79566
[ok  ] url нормализованы: 79566/79566
[ok  ] url проходит фильтры источника: 79566/79566
[ok  ] text заново выделяется из raw_html один в один: 79566/79566
[ok  ] content_hash = SHA-256 текста: 79566/79566
[ok  ] нет страниц-капч: 79566/79566
...
Итог: всё в порядке
```

Полная проверка 79 тысяч документов занимает несколько минут: утилита читает базу потоком, по одному документу, и не держит её в памяти целиком.

Что нашли тесты

Тесты на настоящих страницах нашли две ошибки в выделении текста и ссылок, которые прогон робота не показал. Третью показал прогон.

Контейнер Кино-Театр.Ру не находился. Регулярное выражение искало тег ровно в виде `<div class="...">`, а на сайте он записан как `<div itemprop="text" class="art article_text">`. Контейнер не находился, и в текст попадала вся страница с меню и подвалом. Вдобавок даже найденный контейнер брался до конца страницы, а не до парного `</div>`. У биографий и аннотаций контейнера `art article_text` нет вовсе, текст там лежит в `<div itemprop="description">`.

После исправления текст статьи о «Месте встречи» сократился с 21 280 до 12 852 символов, а биографий актёров — с 8–9 тысяч до 200–400 символов. Отсюда неприятный вывод: 17 документов Кино-Театр.Ру из первого прогона учтены вместе с навигацией, и средний объём текста 21 240 символов по этому источнику завышен. Биографии прошли порог `min_text_length` только за счёт меню. С исправленным выделением

они в корпус не попадают: биографии на этом сайте слишком короткие, чтобы быть документами. Это вторая, после блокировки, причина, по которой источник в итоге заменён на film.ru.

Знак процента в адресе. Уже демонстрационный прогон на настоящих сайтах показал адрес `/wiki/1_%_(фильм)`. Нормализация раскрывала все процентные последовательности, включая `%25` — экранированный сам знак процента, — и адрес становился неоднозначным: голый `%` в пути некорректен. Теперь `%25`, `%2F`, `%3F` и `%23` не раскрываются, на это добавлен тест.

HTML-сущности в ссылках. В атрибуте `href` амперсанд записан как `&`, и без раскрытия сущностей адрес `index.php?title=A&action=edit` превращался в параметр `amp;action`. На Википедии такие ссылки всё равно отсекаются фильтром, но на сайтах с параметрами в адресах это давало бы дубли одного документа.

HTML-сущности в тексте. Первая версия раскрывала только ` `, а остальные сущности удаляла: `«Сталкер» — ... & Стругацких` превращалось в «Сталкер . . . Стругацких» — без кавычек, тире и амперсанда. Сильнее всего это задело film.ru, который пишет кавычки сущностями. Теперь сущности раскрываются, а весь корпус переразобран утилитой `make reparse` из сохранённого HTML, без единого запроса к сайтам.

Когда правила выделения текста меняются, уже скачанные документы не нужно качать заново: в базе хранится «сырой» HTML, и текст переразбирается из него. На этом же построена проверка `make validate`: она заново выделяет текст из `raw_html` и сравнивает с сохранённым.

5 Выводы

Робот написан и работает: он собрал корпус из 79 566 документов с двух источников — русской Википедии и film.ru, — пережил остановки, сон и перезагрузку ноутбука и обрывы сети, а переобкачка находит и обновляет изменившиеся документы. Переобкачка вынесена в отдельную программу и связана с роботом через очередь.

Самое полезное, что дала работа, — это несколько ошибок, которые на бумаге не видны.

Фильтр «что обходить» и фильтр «что сохранять» — это разные фильтры. В первой версии робот складывал в базу оглавления и каталоги: текста в них достаточно, чтобы пройти проверку по длине, и формально они проходили все условия. Понадобился отдельный набор правил, решающий, что считать документом, а что — только источником ссылок.

Без ограничения на то, откуда брать ссылки, корпус перестаёт быть тематическим. Из статьи о фильме ссылки ведут на статьи о городах, годах и профессиях, и через несколько шагов обхода робот собирает уже не кино. Пришлось разрешить сбор ссылок только со страниц категорий и каталогов.

Стандартная библиотека умеет подвести молча. Робот отказывался качать Википедию целиком, помечая все адреса как запрещённые. Причина оказалась в том, что `RobotFileParser.read()` ходит за `robots.txt` через `urllib` со стандартным User-Agent, Викимедиа отвечает на него `403`, а парсер трактует `403` как «запрещено всё». Исправление — качать `robots.txt` той же сессией, что и остальные страницы, и разбирать его через `parse()`.

Адрес нужно нормализовать раньше, чем кажется. Ссылки в HTML Википедии процентно закодированы, и пока адреса не раскрывались, правила с кириллицей по ним не срабатывали: в базу лезли служебные страницы, которые фильтр должен был отсечь.

Сотня документов не показывает того, что покажут десятки тысяч. Демонстрационный прогон прошёл гладко, а полная обкачка вскрыла сразу несколько проблем: продолжение категорий закрыто `robots.txt`, через родительские категории робот уходит из тематики, тысячи красных ссылок тратят запросы впустую, а защита второго источника включается только после сотен запросов. Проверять источник и правила обхода надо на масштабе, а не на образцах.

Сеть ненадёжна, и робот должен это учитывать. Ошибка соединения не означает, что страницы нет. Пока робот считал любую ошибку окончательной, один ночной обрыв интернета стоил двенадцати тысяч адресов. Временные сбои нужно отличать от настоящих ошибок и повторять с паузой.

Второй источник надо выбирать с запасом. Кино-Театр.Ру по пяти страницам выглядел подходящим, но закрылся капчей после пятисот запросов, и блокировка не

снималась сутками. Замена на film.ru оказалась быстрой только потому, что робот с самого начала описывал источник конфигом, а не кодом.

Проверять выделение текста нужно глазами и тестами, а не по счётчикам. Робот отчитывался о сохранённых документах, объёмы выглядели правдоподобно, и то, что у второго источника в текст попадает вся страница, обнаружилось только тестом на настоящей странице: в тексте статьи нашлась строка из подвала сайта.

Недостатки

Робот однопоточный. При паузах в одну и две секунды предел — около полутора запросов в секунду на два источника, а на деле выходит около 0,5: остальное съедает ожидание ответа. Полная обкачка заняла четыре дня. Выбор источника по готовности хоста убирает простой, но не больше: настоящее решение — несколько потоков с очередью на каждый хост, как в схеме URL frontier с лекции.

Корпус Википедии собран неравномерно: из каждой категории берутся только первые 200 статей по алфавиту, так что статьи в начале алфавита представлены лучше. Полный список статей категории без `/w/` не получить; честный путь — дамп Википедии или её API, но это уже не обход страниц.

Около 0,8 % документов Википедии не относятся к кино: категории о фильмах иногда включают основную статью темы («Канада», «Франция»). Отфильтровать их можно по списку категорий самой статьи, который есть в её HTML, — так эта доля и посчитана.

Около пятой части запросов уходит на страницы категорий и списков, которые документами не становятся.

«Сырой» HTML занимает на порядок больше места, чем текст. MongoDB по умолчанию сжимает данные, и на диске база втрое меньше их объёма, но явное сжатие текста словарными методами дало бы больше.

Переобкачка обходит документы по времени последней проверки, то есть одинаково часто для всех. Разумнее было бы учитывать, как часто документ менялся раньше, и проверять стабильные страницы реже, а живые — чаще.

Наконец, робот не защищён от ловушек: календари и страницы с бесконечной пагинацией порождают бесконечную очередь. Пока от этого спасают только ограничение глубины и фильтры адресов.

Список литературы

[1] Маннинг К. Д., Рагхаван П., Шютце Х. *Введение в информационный поиск* — М.: Издательский дом «Вильямс», 2011. — 528 с. — ISBN 978-5-8459-1623-4.

[2] The Robots Exclusion Protocol [Электронный ресурс]. — URL: https://www.robotstxt.org/robotstxt.html (дата обращения: 19.09.2026).

[3] RFC 7232. Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests [Электронный ресурс]. — URL: https://www.rfc-editor.org/rfc/rfc7232 (дата обращения: 19.09.2026).

[4] Film.ru [Электронный ресурс]. — URL: https://www.film.ru/ (дата обращения: 18.10.2026).